Skill progress monitoring

Catching robot failures while they happen

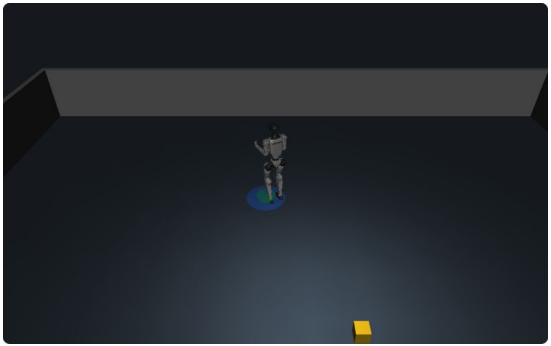
Pinhas Nehoray Aburmad

Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



The problem

- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.

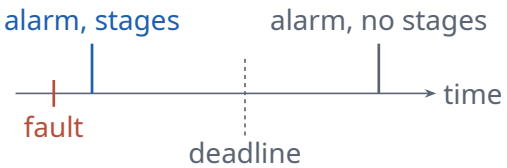


The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

Research question

RQ: Does giving the monitor the **stages** of a skill make it catch failures earlier, and does that improve success?

- **H1:** stage failures are caught **earlier**.
- **H2:** with a deadline, **more tasks succeed**; without one, no change.
- **H3:** it catches failures that **hand-written checks** miss.
- **H0:** for failures every stage shares (falls), **no change**.



Latency ℓ = time from the fault to the alarm.
Re-planning must finish before the deadline.

Related work: two streams

1. Skill description

- **Natural language**
- Formal logic (LTL, STL)
- Videos, demos
- Hand-written code

Our bridge
an LLM writes
LTL rules
+ stages

2. Skill monitoring

- **State machines (automata)**
- LLM reasoning
- VLM / learned detectors
- Hybrid (logic + images)

Most work improves **one** stream. **Ours connects them: the description produces the monitor.**

Stream 1: describing a skill

Way	Examples	What is missing
Natural language	[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu et al., 2023] [nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with LLMs, Cosler et al., 2023] [NL2TL: Transforming Natural Languages to Temporal Logics using LLMs, Chen et al., 2023] [NL2LTL: A Python Package for Converting Natural Language Instructions to LTL Formulas, Fuggitti, Chakraborti, 2023] [GinSign: Grounding Natural Language Into System Signatures for Temporal Logic Translation, English et al., 2025]	Made for planning or writing specs; no stages; words must be tied to sensors
Formal logic (LTL, STL)	[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011] [RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]	An expert writes every rule; flat rules, no stages
Videos / demos	[Learning Task Specifications from Demonstrations, Vazquez-Chanlatte et al., 2018] [RoboCLIP: One Demonstration is Enough to Learn Robot Policies, Sontakke et al., 2023]	Need examples; the result is hard to read or edit
Hand-written code	Our baseline: checks written inside the robot program (Experiment 2)	Someone must foresee every failure

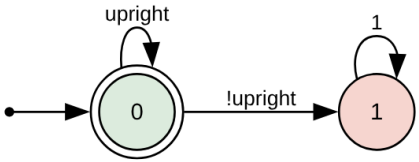
Stream 2: monitoring a skill

Way	Examples	What is missing
State machines	[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011] [ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando et al., 2020] [RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024] [Reelay: Online Temporal Logic Monitoring Framework, Ulus, 2026] [Runtime Verification with State Estimation, Stoller et al., 2011]	No stages: “eventually” waits to the end; rules by hand
LLM reasoning	[REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction, Liu, Bahety, Song, 2023]	Slow; can be wrong; says “wrong”, not “where”
VLM / learned	[AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation, Duan et al., 2025] [FailSafe: Reasoning and Recovery from Failures in VLA Models, Lin et al., 2025] [Foresight: Failure Detection for Long-Horizon Robotic Manipulation, Zhang et al., 2026]	Need training data; black box
Hybrid	[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor et al., 2026]	Facts not human-readable

Our gap: rules made from plain words **+** task stages **+** only the robot’s own sensors.

Background: what is a monitor?

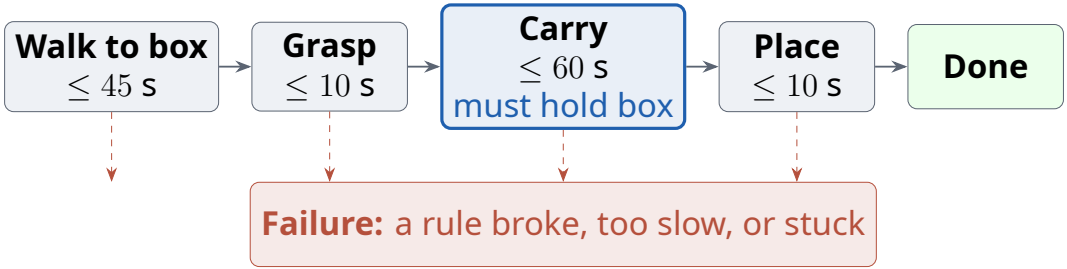
- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.



"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

Our idea: the stages of the task

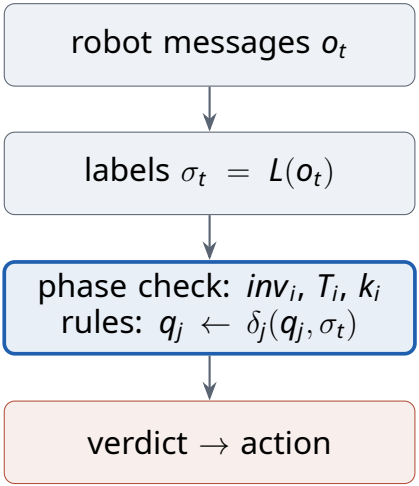
Each stage has its own rules and its own time limit.



While carrying, the robot **must hold the box**. A drop is caught **at once**.

Formal model: specification and monitor

Labelling: each tick, $\sigma_t = L(o_t) \subseteq AP$ (true facts from sensors).
Spec: $S = \langle AP, \Phi, M, \Pi, succ, fail \rangle$: LTL task rules Φ, safety rules M, phases Π.
Phase: $\pi_i = \langle enter_i, inv_i, prog_i, exit_i, T_i, k_i \rangle$.
Monitor state: $m = \langle i, c, v, q_1 \dots q_n \rangle$ (phase, time in it, progress misses, rule states).
Update on σ: every rule steps, $q_j \leftarrow \delta_j(q_j, \sigma)$; **fault** if $\sigma \not\models inv_i$ *or* $c \geq T_i$ *or* $v \geq k_i$; next phase if $\sigma \models exit_i$.



One tick of the monitor (1 Hz).

Model predictions vs. measurements

Property (from the model)	Prediction	Measured
P1. A broken stage invariant fails *at the tick it is seen*.	Drop caught at once.	**1 tick** (20/20)
P2. "Eventually" rules have *no failing state*: any prefix can still succeed.	LTL alone misses the drop until the end.	LTL only: 14 ticks
P3. A stuck stage fails within its time limit T_i.	Failed grasp caught by Grasp's 10 s.	**4 ticks** vs. 19
P4. "Holding" is false before the grasp, so it cannot be a whole-task rule.	Flat spec sees the drop only at the end.	Flat: 14 ticks
P5. Rules every stage shares are in the flat spec too.	Falls: no difference.	1 tick, all versions

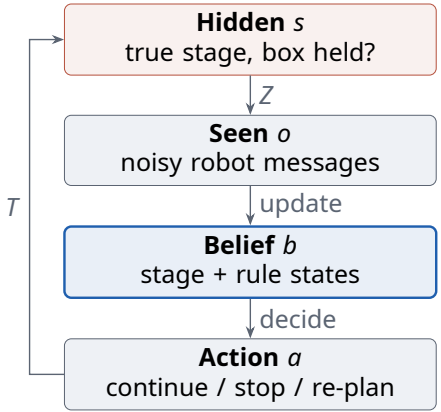
The results follow from the model; they are not a lucky outcome.

The decision view: a POMDP

- **Hidden** s: what really happens (stage, box held?).
- **Seen** o: the robot's messages.
- **Belief** b: our best guess of s.
- **Action** a: continue, stop, or re-plan.

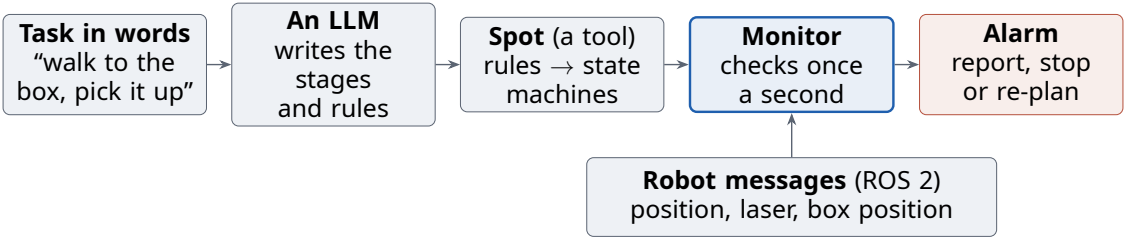
$$b'(s') \;\propto\; \underbrace{Z(o \mid s')}_{\text{does } o \text{ fit } s'?} \sum_{s} \underbrace{T(s' \mid s, a)}_{\text{how } s \text{ changes}} \; b(s)$$

Today: a point belief (stage + rule states).
Value of info: on time needs $\ell + r \le d$.



[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]

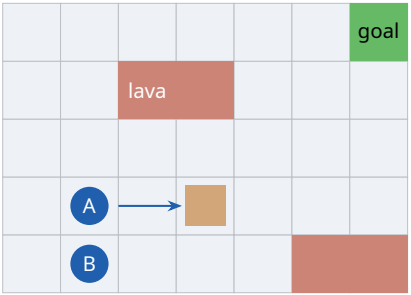
Method: how the monitor is built and run



- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

First test bed: continuous MiniGrid + MAAOS

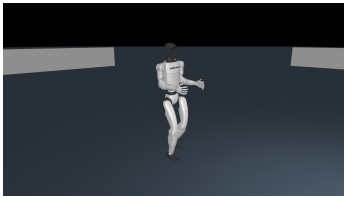
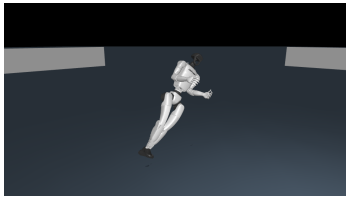
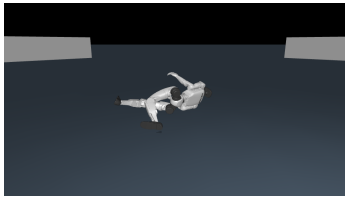
- Box pushing in a **continuous MiniGrid**, with **MAAOS** (Fouzi's multi-agent planner; I integrated it).
- One agent, or two agents pushing together.
- Wrong plan or deadlock: monitor + re-plan **fixed every seed**.
- A free-text **LLM judge: no gain**. It says "wrong", not *where* in the task.
- No false alarms on clean runs.



Two agents (A, B) push a heavy box to the goal and avoid lava.

Test bed: a humanoid robot in MuJoCo

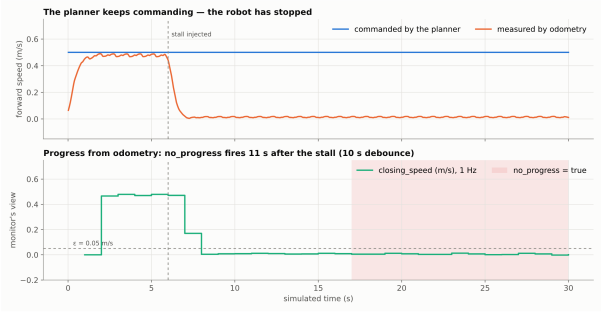
- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.

Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

Experiment 1: a short walk (1800 runs)

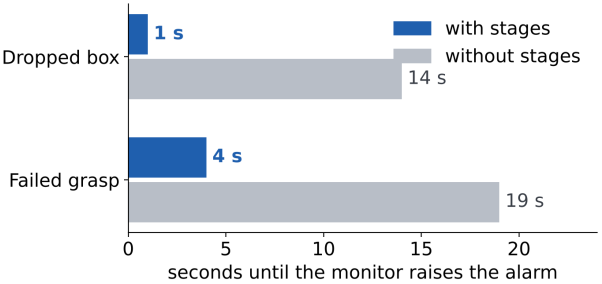
- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors**.
- Without that: only **1 of 15** caught.



The planner still says "go". The robot has stopped. The monitor sees no progress.

Experiment 2: fetch and deliver (3360 runs)

- Four stages.
- Box dropped: caught after **1 s** with stages, 14 s without.
- Grasp failed: **4 s** vs. 19 s.
- Earlier in **all 20** runs.

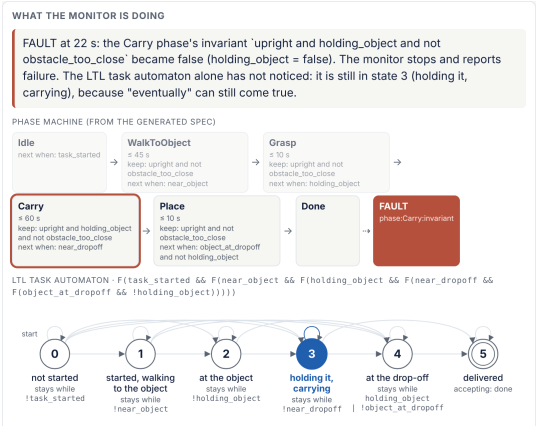


Middle value (median) of 20 runs for each failure.

One run, second by second

- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

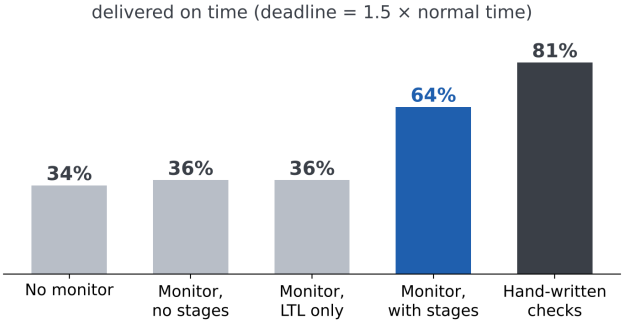
The stage caught it.



From the live demo (one run). The current stage and state are lit.

Does catching early matter?

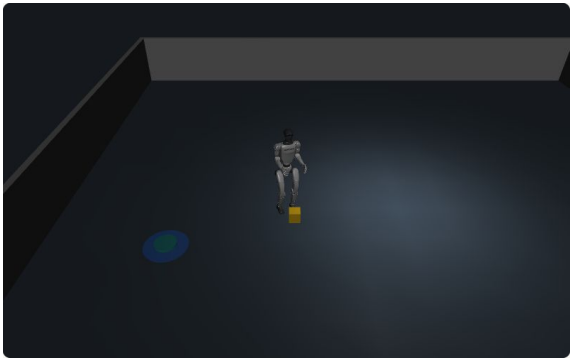
- With no time limit: all monitors ≈ 82–84%.
- With a time limit: **64%** with stages, 36% without.
- Hand-written checks: 81%, but someone must **predict every failure**.



Tasks finished on time, 160 runs per bar. Early knowledge pays when time matters.

Failures nobody planned for

- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: **40 of 40** missed.
- With our monitor: **0 of 40** missed.
- Missed by all: too slow, or a long detour.



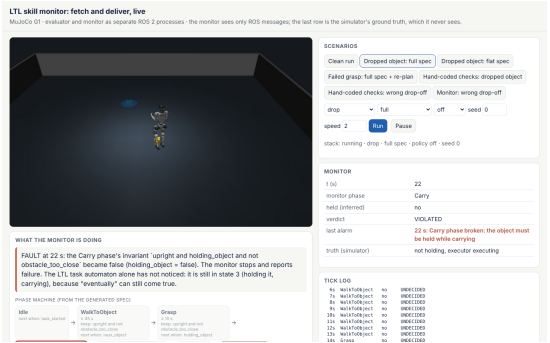
The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

Hypotheses and results

Hypothesis	Result	Verdict
H1: earlier detection	drop 1 vs. 14 s; grasp 4 vs. 19 s; 20/20 runs	**supported**
H2: more on-time success	64% vs. 36% with a deadline; 82% vs. 84% without	**supported**
H3: beyond hand-written checks	40/40 missed $\rightarrow$ 0/40	**supported**
H0: falls unchanged	1 s in every version	as predicted
Not caught	too slow, detours, skipped pick-up	reported

What we learned

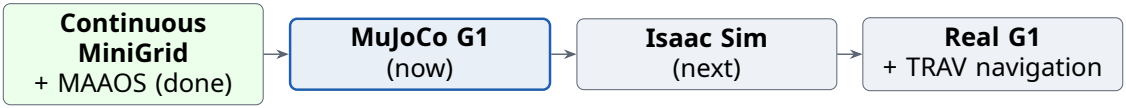
- **Stages catch failures when they happen.**
- Early alarms help **when time matters**.
- Rules come from words, so failures **nobody planned for** are caught too.
- Use the **robot's own sensors**, not the planner.



Live demo, one command:

```
docker run -p 8080:8080
ghcr.io/pinhas019/ltlmonitor-demo
```

To be done



- **Fix what we found:** better time limits; progress toward each stage's goal.
- **More tests:** more tasks, more rule sets, more failures.
- **The full POMDP:** camera, real probabilities, best decisions.

Limits today: grasp and camera are simulated; thresholds not tuned.

References I

[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]. ACM TOSEM 20.

[NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models, Chen, Gandhi, Zhang, Fan, 2023]. EMNLP.

[nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models, Cosler, Hahn, Mendoza, et al., 2023]. CAV.

[AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation, Duan et al., 2025]. ICLR.

[GinSign: Grounding Natural Language Into System Signatures for Temporal Logic Translation, English, Walker, Simon, Ewetz, 2025]. arXiv.

[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando, Cardoso, Fisher, Ancona, et al., 2020]. TAROS.

[NL2LTL: A Python Package for Converting Natural Language Instructions to LTL Formulas, Fuggitti, Chakraborti, 2023]. AAAI (demo).

[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]. Artificial Intelligence 101.

[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor, Hammer, Kapoor, et al., 2026]. arXiv.

[FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models, Lin, Duan, Fang, Fox, et al., 2025]. arXiv.

[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu, Yang, Idrees, Liang, et al., 2023]. CoRL.

[REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction, Liu, Bahety, Song, 2023]. CoRL.

[RoboCLIP: One Demonstration is Enough to Learn Robot Policies, Sontakke, Zhang, Arnold, et al., 2023]. NeurIPS.

[Runtime Verification with State Estimation, Stoller, Bartocci, Seyster, Grosu, Havelund, Smolka, Zadok, 2011]. RV, LNCS 7186.

[Reelay: Online Temporal Logic Monitoring Framework, Ulus, 2026]. arXiv.

[Learning Task Specifications from Demonstrations, Vazquez-Chanlatte, Jha, Tiwari, Ho, Seshia, 2018]. NeurIPS.

[RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]. STTT 26.

[Foresight: Failure Detection for Long-Horizon Robotic Manipulation with Action-Conditioned World Model Latents, Zhang, Lu, Wang, Kang, et al., 2026]. arXiv.